Artificial Intelligence and Applications Lab Practical File

Sree Saicharan Vadapalli
Information Technology
ET23BTIT133

April 3, 2026

Contents

1 Analysis of Major Projects in AI **3**

1.1 Deep Blue . . . 3

1.1.1 Historical Background and Development . . . 3

1.1.2 Algorithmic Design and Evaluation Strategy . . . 4

1.1.3 System Architecture . . . 5

1.1.4 Performance, Limitations and Impact . . . 6

1.2 Chinook . . . 7

1.2.1 Historical Background and Development . . . 7

1.2.2 Computational Characteristics of English Draughts . . . 7

1.2.3 Algorithmic Design and Evaluation Function . . . 8

1.2.4 System Architecture . . . 8

1.2.5 Performance, Limitation and Impact . . . 9

1.3 ALVINN . . . 9

1.3.1 Historical Background and Development . . . 10

1.3.2 Network Architecture and Learning Mechanism . . . 10

1.3.3 Performance, Limitations, and Impact . . . 11

1.4 ELIZA . . . 11

1.4.1 Historical Background and Development . . . 12

1.4.2 Algorithmic Design . . . 12

1.4.3 Performance, Limitations, and Impact . . . 12

1.5 ChatGPT . . . 13

1.5.1 Historical Background and Development . . . 13

1.5.2 Architectural Design . . . 14

1.5.3 Performance, Limitations, and Impact . . . 14

1.6 Google Translate . . . 15

1.6.1 Historical Background and Development . . . 15

1.6.2 Algorithmic Design . . . 15

1.6.3 Performance, Limitations, and Impact . . . 16

1.7 Gemini . . . 16

1.7.1 Historical Background and Development . . . 17

1.7.2 Architectural Design . . . 17

1.7.3 Performance, Limitations, and Impact . . . 17

1.8 Nano Banana . . . 18

1.8.1 Historical Background and Development . . . 18

1.8.2 Algorithmic Design . . . 18

1.8.3 Performance, Limitations, and Impact 19

2 8 Puzzle Problem **20**
2.1 Problem Statement 20
2.2 Theory 20
2.3 Implementation 21
2.4 Output 22
2.5 Conclusion 23

3 Water Jug Problem **24**
3.1 Problem Statement 24
3.2 Theory 24
3.2.1 State Representation 24
3.2.2 Allowed Operations 24
3.2.3 Search Approach 25
3.2.4 Problem Characteristics 25
3.3 Implementation 25
3.4 Output 26
3.5 Conclusion 26

4 Basic Medical Expert System in Prolog **27**

5 Append Two Lists in Prolog **28**

6 Reverse a List in Prolog **29**

7 Finding n^th Element in Prolog **30**

8 Member Check Program in Prolog **31**

9 Fibonacci Series **32**

10 Factorial Series **33**

11 Tower of Hanoi **34**

12 Tic-Tac-Toe using Minimax Algorithm **35**

Practical 1

Analysis of Major Projects in AI

1.1 Deep Blue

In May 1997, IBM's Deep Blue, a chess-playing supercomputer designed by computer scientist Feng-hsiung Hsu, defeated the reigning world chess champion Garry Kasparov under standard tournament controls. Deep Blue disproved the assumption that humans were the strongest chess-playing entities. It was the first computer to win a game and a match against a reigning world champion under standard time controls. Deep Blue was a breakthrough in computer chess and high-performance search algorithms. IBM's first supercomputer, Stretch had a processing speed of 500,000 floating-point operations (FLOP). Deep Blue was capable of evaluating 200 *million* positions per-second, achieving a processing speed of 11.38 *billion* FLOPs per-second. Deep Blue heavily relied on **game-tree search** and hand-made evaluation heuristics instead of modern machine learning techniques. IBM's primary rationale behind creating a chess-playing supercomputer was to determine whether a computational search system could outperform human expertise.

1.1.1 Historical Background and Development

Feng-hsiung Hsu, a doctoral student at Carnegie Mellon University, began development of a chess-playing supercomputer under the name of ChipTest and even won the North American Computer Chess Championship in 1987. In 1988, Hsu and his team created a successor to ChipTest and named it "Deep Thought". Deep Thought, the reigning world computer chess champion of its time, lost a two-game exhibition match to the Russian grandmaster Garry Kasparov in 1989. After the loss, IBM renamed the chess machine as "Deep Blue", which was a play on IBM's nickname "Big Blue" at the time. Hsu and Murray Campbell hired Joel Benjamin to assist with the preparations for Deep Blue's matches against Kasparov. Deep Blue went on to play in the eight world computer chess championship and lost to Fritz (German Chess Program) in round five despite playing as white. Deep Blue played Kasparov twice more after Deep Thought's loss in 1989, losing both. Before it faced Kasparov again in May 1997, IBM upgraded Deep Blue's hardware, doubling its speed. During the six-game rematch, Deep Blue secured a victory in the deciding game with the score $3\frac{1}{2}$–$2\frac{1}{2}$ against Kasparov. Thus, it became the first computer system to defeat the reigning world champion in a match under standard time controls. The version of Deep Blue that was victorious against Kasparov searched to a depth of six to eight moves and around 20 moves in some

situations. Kasparov demanded a rematch, but IBM had dismantled Deep Blue after its victory and refused the rematch, which remains a subject of controversy in chess community. Since Deep Blue's victory, computer scientists have developed software for complex board games, the AlphaGo series (AlphaGo, AlphaGo Zero, AlphaZero) defeated top Go players in 2016–2017.

1.1.2 Algorithmic Design and Evaluation Strategy

Unlike modern learning-based systems, Deep Blue relied on deterministic search and expert-designed heuristics instead of statistical training. It derived its strength primarly from a large and complex game-tree search combined with handcrafted evaluation function (set of rules and mathematical weights). Before the modern AI era, computers didn't "learn" good positions in the traditional sense. They're mostly considered to be a form of Symbolic AI or Good Old Fashioned AI (GOFAI).

Tree Search

In computer science, a search tree is a dynamic, hierarchical data structure. It's represented as a directed graph of nodes (also called vertices or states) connected by directed edges (also called arcs or state transitions). A chess game can be represented as a tree search problem because a chess game is deterministic, has perfect information and is zero-sum. The states are the particular arrangements of pieces on the chess board and the available actions are the possible legal chess moves for the current player in a specfic arrangement. In the game of chess, nodes represent the altering white and black "to move" chess positions and directed edges represent the alternative white and black moves.

Evaluation Function

Shannon's paper describes evaluation function as the primary ingredient in a chess-playing program. It is a type of function that takes in the current state of the game and reduces it to an estimated scalar (numerical) value of the state that is used for evaluating that state.

$$\text{Score} = \sum_i w_i \, f_i(\text{position})$$

where:

$$f_i \text{ represents positional features,}$$

$$w_i \text{ represents corresponding weights.}$$

Minimax and Alpha-Beta Pruning

Deep Blue implmented a minimax search framework apt for deterministic, zero-sum and perfect-information games such as chess. The minimax program made the following assumptions:

- The maximizing player selects moves to maximize the evaluation score.
- The minimizing player (opponent) selects moves to minimize the score.

In simple terms, Minimax looks ahead as many steps as the computing power allows for each move and examine all the possible moves the opponent could make in each of the opponent's future turns, given that the original move is made. It computes the maximum loss that the opponent could induce via their moves and attempts to choose the move that minimizes it the most.The formal recursive definition of the minimax algorithm is:

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is terminal} \\ \max\limits_{a \in \text{Actions}(s)} V(\text{Result}(s,a)) & \text{if } s \text{ is Max node} \\ \min\limits_{a \in \text{Actions}(s)} V(\text{Result}(s,a)) & \text{if } s \text{ is Min node} \end{cases}$$

where:

$s \in S$	game state
$S_T \subseteq S$	set of terminal states
$S_{\text{Max}} \subseteq S$	states where Max plays
$S_{\text{Min}} \subseteq S$	states where Min plays
$A(s)$	legal actions in state s
$T(s,a)$	transition function
$U(s)$	utility function

Additionally, Deep Blue utilized Alpha-Beta pruning to elimate branches that had minimal to no effect on the final decision because the size of a full chess game tree ($Branching\ Factor \approx 35$ and $Shannon\ Number \approx 10^{120}$) makes it computationally unmanageable. Alpha-beta pruning maintains two bounds:

- **Alpha** (α)**:** Best value found so-far for the maximizing player.
- **Beta** (β)**:** Best value found so-far for the minimizing player.

Exploration of a branch is pruned or considered unnecessary when $\alpha \geq \beta$. This substantially reduces the **effective branching factor** and enables deeper search. It is important to note that alpha-beta pruning is implemented using Depth-First Search (DFS), meaning it only one path from the root to the current position needs to be kept in memory. It introduces α and β values to the minimax value.

Transposition Tables

Deep Blue used transposition tables to store previously evaluated positions because many chess positions can be reached through a different order of moves. Deep Blue used Zobrist hashing to hash the positions, allowing constant-lookup time and avoiding redundant computation.

1.1.3 System Architecture

Deep Blue's computational dominance was enabled by specialized hardware and large-scale parallelization. The system represented a co-design of software search algorithms and domain-specific hardware acceleration. The 1997 version of Deep Blue consisted of:

1. 30 IBM RS/6000 SP nodes

2. 480 custom VLSI chess chips
3. High-speed interconnect for distributed computation

Each node had custom chess processors that were capable of: generating legal moves in hardware, evaluating positions and performing search acceleration operations. Unlike general-purpose CPUs, these chips were specifically designed for chess computation. This domain specific design allowed Deep Blue to outperform pure software-based systems.

Parallel Search Strategy

It implemented parallel search strategy using a master-slave architecture. Different processors could explore different legal moves from root position simultaneously, which reduced the overall time taken to reach a decision. This is also called **root level parallelism**. Search trees were dynamically partitioned among processors and synchronization was essential to maintain correct alpha-beta bounds across the nodes. Efficient synchronization prevents; redundant exploration, bound inconsistencies, communication overhead.

1.1.4 Performance, Limitations and Impact

Performance

Deep Blue's performance was primarily driven by large-scale parallel search and hardware acceleration. The 1997 system evaluated approximately 200 million positions per second and achieved a peak performance of around 11.38 billion floating-point operations per second. In practical play, it typically searched to depths of six to eight moves in complex positions and up to twenty moves in tactical lines. Its strength did not come from floating-point computation alone, but from efficient alpha–beta pruning, strong move ordering, selective search extensions, and the use of custom VLSI chess chips across multiple processing nodes.

Limitations

Despite its success, Deep Blue was a highly specialized system designed exclusively for chess. It did not possess learning capabilities and could not adapt or improve through self-play. Its evaluation function was manually engineered and relied on predefined heuristics rather than statistical training. Furthermore, its strength depended heavily on massive computational resources and domain-specific hardware. Without this specialized infrastructure, its performance would have been significantly reduced.

Impact

Deep Blue's victory over Garry Kasparov marked a major milestone in artificial intelligence and computer chess. It demonstrated that large-scale computational search could surpass human expertise in a complex strategic domain. Later systems, such as AlphaZero, introduced learning-based approaches that fundamentally changed how game-playing AI systems were designed.

1.2 Chinook

In 1996, Microsoft introduced Chinook, a computer checkers program developed at the University of Alberta under the leadership of computer scientist Jonathan Schaeffer. Chinook challenged and later defeated the reigning world checkers champion Marion Tinsley, marking the first time a computer program competed for and won a human world championship title in any game. Unlike earlier systems that relied primarily on heuristic evaluation, Chinook combined high-performance game-tree search with large, expertly constructed endgame databases containing precomputed perfect-play positions. By 2007, after years of large-scale retrograde analysis, the team solved the game of checkers (English draughts), proving that perfect play from both sides results in a draw. Chinook demonstrated that exhaustive search, domain-specific heuristics, and massive endgame computation could not only rival but mathematically surpass human expertise in deterministic, perfect-information games.

1.2.1 Historical Background and Development

In 1989, Jonathan Schaeffer and his research team at the University of Alberta began developing Chinook, a computer program designed to compete at the highest levels of professional checkers. The project quickly advanced, and in 1992 Chinook earned the right to challenge reigning world champion Marion Tinsley. Although it narrowly lost that initial match, the event marked the first time a computer program competed for a human world championship title in any game.

A rematch was held in 1994, but Tinsley withdrew due to health reasons. Chinook subsequently competed against Don Lafferty for the vacant title and won, becoming the first computer program to win a human world championship in a game of intellectual skill. Following its competitive success, the Chinook team continued refining the program with the long-term objective of fully solving checkers.

After years of computational analysis and database construction, the team announced in 2007 that the game of checkers had been weakly solved, proving that perfect play from both sides results in a draw. Chinook's competitive career was formally retired after this milestone, concluding nearly two decades of sustained research and development.

1.2.2 Computational Characteristics of English Draughts

The version of checkers addressed by Chinook is formally known as English draughts. From a computational perspective, it is a deterministic, perfect-information, two-player zero-sum game. Each of these properties has direct algorithmic implications.

These characteristics reduce the problem to adversarial search over a fully observable state space. There is no hidden information, no probability modeling, and no stochastic branching. The entire game can, in principle, be represented as a finite directed graph where; nodes represent legal positions on the board, edges represent legal moves and terminal nodes denote a win or loss or draw. This makes checkers mathematically well-defined and suitable for exhaustive analysis.

State Space and Game Tree Size

- **Estimated total legal positions** $\approx$ order of 10^{20}

- **Average branching factor** $\approx$ 8–10 moves per-position
- **Maximum game length:** bounded due to repetition and capture rules.

Although 10^{20} positions is enormous, it is dramatically smaller than chess ($\approx 10^{42}$ estimated positions) and vastly smaller than Go ($\approx 10^{170}$). This relative tractability is what made checkers a viable candidate for complete solution.

Why Checkers Was Solvable

The key insight is not that checkers is "simple," but that it is finite and computationally bounded. A game is solvable if it meets the following conditions:

1. Its state space can be fully enumerated or decomposed
2. Terminal outcomes can be propagated backward
3. Storage and compute resources are sufficient

Checkers met these conditions decades before chess or Go could realistically do so. The combination of moderate branching factor, forced-move constraints, and reducible endgame complexity made it the first non-trivial classical board game to be weakly solved at world-championship scale.

1.2.3 Algorithmic Design and Evaluation Function

The algorithmic foundation of Chinook was a depth-limited minimax search enhanced with alpha–beta pruning and iterative deepening to efficiently explore the adversarial game tree under tournament time constraints. To mitigate redundant computation, the system employed large transposition tables indexed via Zobrist hashing, enabling constant-time retrieval of previously evaluated positions. Move ordering heuristics were used to maximize pruning effectiveness, particularly in tactically forced capture sequences. The evaluation function combined material balance, piece advancement, mobility, center control, tempo considerations, and structural formations, with parameters refined through extensive expert consultation and empirical tuning rather than machine learning. In late-game scenarios, heuristic evaluation was replaced by exact lookup into precomputed endgame databases generated through retrograde analysis, ensuring mathematically optimal play once positions entered solved regions. This hybrid design—forward search guided by domain-specific heuristics and terminated by exact database resolution—allowed Chinook to approximate optimal play in the middlegame while guaranteeing perfect decisions in endgame configurations.

1.2.4 System Architecture

The system architecture of Chinook was designed around high-performance sequential search augmented by large-scale persistent storage for endgame databases. The core engine executed on conventional high-end workstations, relying on optimized C implementations, efficient bitboard representations, and tightly controlled memory management to maximize search throughput per clock cycle. A layered architecture separated move generation, search control, evaluation, and database access, allowing clean integration of exact endgame lookups into the forward search pipeline. Transposition tables were maintained in RAM for rapid access, while larger solved-position databases were stored on disk with compressed indexing schemes

to balance storage constraints and lookup latency. The architecture emphasized space–time tradeoffs: heavy precomputation and storage reduced runtime computation during tournament play. Unlike massively parallel supercomputing systems, Chinook's design prioritized algorithmic efficiency, caching, and structured decomposition of the game space over raw hardware scaling, reflecting the engineering goal of solving a finite adversarial domain within practical computational limits.

1.2.5 Performance, Limitation and Impact

Performance

Chinook achieved superhuman performance through deep alpha–beta search combined with exact endgame database lookups. In tournament play, it consistently evaluated millions of positions per move while leveraging cached transpositions to avoid redundant computation. Its strength was most pronounced in late-game scenarios, where database access guaranteed mathematically optimal decisions. This hybrid of heuristic-guided search and perfect endgame resolution enabled Chinook to compete at, and ultimately surpass, world-championship human performance.

Limitation

Despite its dominance, Chinook was fundamentally domain-specific and non-generalizable. Its architecture depended on the finite and relatively constrained state space of English draughts; the approach does not scale to vastly larger games such as chess or Go without prohibitive storage and computation costs. The system required extensive precomputation and long development cycles, making it impractical for dynamic or uncertain environments. It was an engineered solver, not an adaptive learning system.

Impact

Chinook's 2007 weak solution of checkers established the first complete resolution of a non-trivial classical board game at championship scale. The project demonstrated that exhaustive computation, when paired with rigorous state-space decomposition, can fully solve structured adversarial domains. Beyond competitive play, it provided a proof-of-concept for large-scale retrograde analysis and influenced subsequent research in computational game theory, search optimization, and formal game solving.

1.3 ALVINN

In 1989, researchers at Carnegie Mellon University developed ALVINN (Autonomous Land Vehicle In a Neural Network), one of the earliest end-to-end neural network systems for autonomous driving. Created by Dean Pomerleau, ALVINN was designed to learn steering control directly from camera images using supervised training, rather than relying on manually programmed rules or symbolic reasoning. The system was deployed on the Navlab research vehicle and demonstrated the ability to steer a car autonomously on real roads at highway speeds. Unlike search-based game-playing systems of its era, ALVINN represented a shift toward connectionist approaches, mapping raw sensory input to motor output through

learned internal representations. It marked an early and influential step toward modern deep learning–based autonomous driving systems.

1.3.1 Historical Background and Development

The development of ALVINN (Autonomous Land Vehicle In a Neural Network) emerged during the late 1980s within the autonomous vehicle research program at Carnegie Mellon University. At the time, most autonomous navigation systems relied on symbolic AI, rule-based perception pipelines, and handcrafted feature extraction. These systems required explicit modeling of lane markings, road geometry, and vehicle kinematics, resulting in brittle performance under real-world variability.

Under the direction of Dean Pomerleau, ALVINN pursued a fundamentally different paradigm: learning a direct mapping from camera input to steering commands using a neural network. Development began around 1988–1989 as part of the Navlab autonomous vehicle project. The system was trained using supervised learning, where human driving examples were recorded and used to adjust network weights via backpropagation. Rather than encoding explicit rules for lane detection, ALVINN learned internal representations sufficient to produce appropriate steering angles. The system was deployed on CMU's Navlab vehicle platform and demonstrated sustained autonomous driving on public highways, including extended stretches where the neural network controlled steering at typical traffic speeds. Subsequent iterations incorporated additional sensors and refinements to improve robustness under varying lighting and road conditions. ALVINN's development marked an early and influential shift from symbolic robotics toward data-driven control systems, anticipating the end-to-end learning architectures that would dominate autonomous driving research decades later.

1.3.2 Network Architecture and Learning Mechanism

ALVINN's network architecture consisted of a three layer backpropagation neural network.

Input Layer

ALVINN processed gray-scale images of 30×32 where each image was flattened into a single vector of 960 dimension (30×32). The input layer primarily received data from:

- **Cameras** that provide images of the road.
- **Laser Range Finders** that offer distance measurements from objects in proximity.

Hidden Layer

ALVINN's hidden layer consisted of approximately 30 neurons, intermediate between the sensory input and steering output. Each node in the hidden layer has weighted connections to all the 960 sensory input nodes, which allows the network to learn patterns present in an image.

During training, the hidden layer learned internal feature representations such as road boundaries, lane workings, and variation in road texture from the input images. ALVINN did not rely on manually designed algorithms and learned the patterns by adjusting its weights during training.

Output Layer

The ALVINN output layer consists of 45 units, each representing a possible steering direction for the vehicle. The center unit signals to go straight, while units to the left and right correspond to progressively sharper left and right turns. During training, the network learns using a hill-shaped activation centered on the correct unit, which indicates the curvature needed to reach the center of the road about 7 meters ahead. This design allows smooth and precise steering control based on the processed sensory inputs.

1.3.3 Performance, Limitations, and Impact

Performance

ALVINN demonstrated sustained autonomous steering on real public highways at speeds up to 55 mph, controlling the Navlab vehicle for stretches of several kilometers without human intervention. Despite its shallow architecture of only 30 hidden units, it generalized adequately across varying road textures and lighting conditions encountered during deployment. The network processed each input frame and produced a steering output in real time, with forward inference being computationally inexpensive due to the small number of parameters.

Limitations

ALVINN's primary limitation was poor generalization to road conditions not represented in the training data. Because the system learned a direct input-to-output mapping without any explicit geometric or semantic understanding of the environment, it was brittle when encountering novel scenarios such as intersections, sharp curves, or adverse weather. The network only controlled steering; throttle and braking required separate human or programmatic intervention. Additionally, the 30×32 grayscale input provided very limited spatial resolution, constraining perception quality significantly.

Impact

ALVINN was among the first systems to demonstrate that a neural network could learn to drive a real vehicle on public roads directly from raw sensor input, without hand-engineered feature extraction or symbolic rule systems. It established end-to-end learning as a viable paradigm for sensorimotor control. Systems such as NVIDIA's DAVE and later PilotNet (2016) explicitly built upon the end-to-end learning philosophy pioneered by ALVINN, validating its design philosophy nearly three decades later.

1.4 ELIZA

In 1966, Joseph Weizenbaum at the Massachusetts Institute of Technology developed ELIZA, one of the earliest natural language processing programs capable of simulating a conversation with a human user. ELIZA operated by pattern matching user input against a set of predefined script rules and generating contextually appropriate responses by transforming and reflecting the user's own statements back at them. Its most well-known script, **DOCTOR**, simulated a Rogerian psychotherapist. ELIZA did not possess any understanding of language or context;

its apparent intelligence was entirely an artifact of clever pattern substitution, a phenomenon Weizenbaum later termed the *ELIZA effect*.

1.4.1 Historical Background and Development

Weizenbaum developed ELIZA between 1964 and 1966 as a demonstration of the superficiality of human–computer communication. The DOCTOR script was chosen specifically because Rogerian therapy relies heavily on non-directive, reflective questioning, which naturally maps to ELIZA's pattern-matching strategy. Weizenbaum was reportedly alarmed by how readily users, including his own secretary, attributed genuine understanding and empathy to the program despite being aware of its mechanical nature. This reaction prompted him to write *Computer Power and Human Reason* (1976), a critique of the anthropomorphization of computing systems. ELIZA was implemented in MAD-SLIP on the IBM 7094 and later reimplemented in numerous languages, making it one of the most widely reproduced early AI programs.

1.4.2 Algorithmic Design

ELIZA's operation was entirely rule-driven and required no learning or statistical inference. Its core mechanism consisted of three components: a pattern matcher, a decomposition–reassembly rule set, and a script interpreter.

Pattern Matching and Decomposition

Each input sentence was scanned for keywords ranked by a priority weight. Upon identifying the highest-priority keyword, ELIZA applied a decomposition rule that parsed the sentence into syntactic fragments using wildcard matching. A reassembly rule then recombined those fragments into a response, often performing simple pronoun substitution (e.g., "I" → "you"). For example, the input *"I am feeling sad"* might match the keyword *I AM*, decompose into the fragment *feeling sad*, and reassemble as *"Why do you say you are feeling sad?"*.

If no keyword matched, ELIZA would fall back to a set of context-free default responses such as *"Please go on"* or *"Tell me more about that"*, maintaining the illusion of engagement without any semantic processing.

Script Architecture

The rules were encoded in a separate, interchangeable script file rather than hardcoded into the program. This separation of the interpreter from the knowledge base allowed different conversational personas to be loaded by supplying a different script. The DOCTOR script contained approximately 200–250 decomposition and reassembly rules covering common emotional and self-referential statements.

1.4.3 Performance, Limitations, and Impact

Performance

Within the narrow domain of the DOCTOR script, ELIZA produced responses that were coherent and contextually plausible for short exchanges, creating a convincing illusion of

understanding particularly when users were emotionally invested. However, it had no memory of prior turns beyond the immediately preceding statement and could not maintain topical consistency across longer dialogues.

Limitations

ELIZA possessed no semantic understanding whatsoever. Its responses were generated purely by syntactic pattern matching and string transformation, with no internal representation of meaning, intent, or world knowledge. It could not handle ambiguous phrasing, metaphor, or statements that did not match any rule in its script. The system was entirely static and could not learn from or generalize beyond its predefined rule set.

Impact

ELIZA was foundational to both natural language processing and human–computer interaction research. It introduced the concept of a **chatbot** and directly inspired subsequent dialogue systems. The ELIZA effect—the tendency of users to over-attribute understanding to conversational programs—remains a relevant concern in the evaluation and deployment of modern large language models. ELIZA also influenced the design of later rule-based systems such as PARRY (1972) and the broader tradition of scripted dialogue agents.

1.5 ChatGPT

In November 2022, OpenAI publicly released ChatGPT, a large language model–based conversational AI system built on the GPT (Generative Pre-trained Transformer) architecture. Unlike ELIZA's rule-based approach, ChatGPT generates responses by predicting the most probable continuation of a conversation using a transformer neural network trained on a large corpus of text and fine-tuned using Reinforcement Learning from Human Feedback (RLHF). ChatGPT demonstrated unprecedented fluency and general-purpose capability across tasks including coding, writing, analysis, and question answering, reaching one million users within five days of launch and fundamentally shifting public perception of what language models could accomplish.

1.5.1 Historical Background and Development

ChatGPT is built on the GPT series of models developed by OpenAI, beginning with GPT-1 (2018), which introduced the **pre-train then fine-tune** paradigm for language models. GPT-2 (2019) scaled this approach and was initially withheld from full public release due to concerns about misuse. GPT-3 (2020) demonstrated that scaling model parameters to 175 *billion* produced emergent few-shot and zero-shot capabilities not seen in smaller models. ChatGPT was released as a fine-tuned variant of GPT-3.5, specifically adapted for dialogue using RLHF, where human trainers ranked model outputs to train a reward model that subsequently guided policy optimization via Proximal Policy Optimization (PPO). GPT-4 was released in March 2023 as a multimodal successor with substantially improved reasoning and factual accuracy.

1.5.2 Architectural Design

Transformer Architecture

ChatGPT is based on the decoder-only transformer architecture introduced in *Attention Is All You Need* (Vaswani et al., 2017). The core mechanism is **scaled dot-product self-attention**, which allows every token in the input sequence to attend to every other token, enabling the model to capture long-range dependencies without the sequential bottleneck of recurrent networks. Multi-head attention runs this operation in parallel across multiple attention heads, allowing the model to simultaneously attend to information from different representational subspaces. The output of all heads is concatenated and projected back to the model dimension. Stacking many such transformer blocks with feed-forward sublayers, residual connections, and layer normalization produces the deep network used for language modeling.

Pre-training Objective

During pre-training, the model is trained on a **next-token prediction** (autoregressive language modeling) objective over a large text corpus. Given a sequence of tokens, the model learns to predict the next token from all preceding tokens. This objective forces the model to internalize syntactic, semantic, and factual structure present in the training data without any task-specific supervision.

Reinforcement Learning from Human Feedback

After supervised fine-tuning on human-written demonstration data, a separate **reward model** is trained on human preference rankings of model outputs. The language model is then further optimized using Proximal Policy Optimization (PPO) to maximize the reward signal, with a penalty term that prevents the policy from deviating too far from the supervised baseline. This three-stage pipeline—pre-training, supervised fine-tuning, and RLHF—aligns model outputs with human preferences for helpfulness, harmlessness, and honesty.

1.5.3 Performance, Limitations, and Impact

Performance

ChatGPT demonstrated strong performance across code generation, mathematical reasoning, creative writing, and multi-turn dialogue. GPT-4 subsequently achieved near-human performance on several professional benchmarks including the bar exam, SAT, and GRE. The global receptive field of the attention mechanism allows the model to handle long-context dependencies effectively.

Limitations

ChatGPT is prone to **hallucination**—confidently generating factually incorrect statements—because it optimizes for plausible token sequences rather than verifiable truth. It has no persistent memory across sessions by default and lacks grounding in real-time information. The model is also sensitive to prompt phrasing, producing inconsistent outputs for semantically equivalent inputs. Bias inherited from training data remains a persistent concern.

Impact

ChatGPT accelerated mainstream adoption of large language models and triggered a rapid wave of competing systems from Google, Meta, Anthropic, and others. It established RLHF as the dominant alignment technique for conversational AI and prompted significant regulatory and ethical debate around AI-generated content, academic integrity, and labor displacement.

1.6 Google Translate

Google Translate is a multilingual machine translation service developed by Google, first launched in April 2006. It initially operated using a statistical machine translation (SMT) approach and transitioned to a neural machine translation (NMT) architecture in November 2016 with the deployment of the Google Neural Machine Translation (GNMT) system. As of 2024, Google Translate supports over 100 languages and processes billions of words daily, making it the most widely used machine translation system in the world.

1.6.1 Historical Background and Development

Google Translate was originally built on phrase-based statistical machine translation, a technique that learns translation probabilities from large bilingual corpora without any explicit linguistic rules. The SMT system decomposed source sentences into phrases and computed the most probable target-language phrase sequence using a log-linear model combining a translation model and a language model. While effective for short inputs, SMT produced grammatical inconsistencies and handled long-range dependencies poorly.

In 2016, Google replaced the SMT backend with GNMT, a sequence-to-sequence model based on stacked Long Short-Term Memory (LSTM) networks with an attention mechanism, producing a significant measured improvement in translation quality. Subsequent iterations moved toward Transformer-based models for higher parallelism and better long-range dependency modeling. Google also introduced **zero-shot translation**, where the system could translate between language pairs it had never been directly trained on by routing through a shared multilingual representation space.

1.6.2 Algorithmic Design

Statistical Machine Translation (Early Architecture)

The original SMT system modeled translation as a probabilistic search problem. Given a source sentence, the goal was to find the target sentence that maximized the product of a **translation model** (how likely the source is a translation of the target) and a **language model** (how fluent the target sentence is). In practice this was implemented as a log-linear combination of feature functions optimized using minimum error rate training (MERT). The system decomposed source sentences into phrases and computed the most probable target-language phrase sequence, but handled long-range syntactic dependencies poorly.

Neural Machine Translation and Attention

The GNMT system replaced the phrase-based pipeline with an **encoder–decoder** architecture. The encoder read the source sequence token by token, producing a sequence of hidden

states. The decoder generated the target sequence one token at a time, using an **attention mechanism** (Bahdanau et al., 2015) to compute a weighted sum over all encoder hidden states at each decoding step. This attention weight indicated which parts of the source sentence were most relevant when generating each target token, allowing the model to handle long sentences and non-monotonic word order far better than fixed-length encoder representations. Modern Google Translate is based on Transformer encoders and decoders, replacing recurrence with multi-head self-attention for higher parallelism and better long-range dependency modeling.

1.6.3 Performance, Limitations, and Impact

Performance

The transition from SMT to NMT reduced translation errors by approximately 60% on several language pairs as measured by human evaluators at Google. For high-resource pairs such as English–French and English–German, modern Google Translate produces translations of near-professional quality. The system supports document, image, and speech translation in addition to plain text, processing input in real time.

Limitations

Translation quality degrades significantly for low-resource languages where parallel training data is scarce. The system struggles with idiomatic expressions, domain-specific terminology, and languages with complex morphology or free word order. Cultural nuance and pragmatic meaning are frequently lost, and translations of long documents can lose coherence across sentences due to the absence of global document-level context.

Impact

Google Translate substantially lowered the barrier to cross-lingual communication for billions of users and drove widespread adoption of neural machine translation across the industry. The demonstration of zero-shot translation provided early evidence that neural networks could form language-agnostic semantic representations, a finding with broad implications for multilingual representation learning.

1.7 Gemini

In December 2023, Google DeepMind released Gemini, a family of natively multimodal large language models designed to reason across text, images, audio, video, and code within a unified architecture. Gemini succeeded the PaLM series as Google's flagship foundation model and was released in three capability tiers: Ultra, Pro, and Nano, targeting data center, general-purpose, and on-device deployment respectively. Unlike earlier multimodal systems that coupled separate vision and language encoders, Gemini was trained end-to-end on interleaved multimodal data from the outset, enabling tighter integration of cross-modal reasoning.

1.7.1 Historical Background and Development

Gemini was developed jointly by Google Brain and DeepMind following their organizational merger into Google DeepMind in 2023, as a direct competitive response to the rapid adoption of GPT-4. Google had previously demonstrated strong language modeling through BERT (2018), T5 (2019), and PaLM (2022), and multimodal capabilities through Flamingo (2022) and PaLM-E (2023). Gemini consolidated these efforts: rather than combining specialist models post-hoc, it was designed from the ground up to natively process multiple modalities. Gemini Ultra became the first model to surpass human expert performance on the Massive Multitask Language Understanding (MMLU) benchmark at the time of its release. Gemini 1.5 followed in early 2024, introducing a Mixture-of-Experts (MoE) architecture and a context window of up to 1 *million* tokens.

1.7.2 Architectural Design

Multimodal Architecture

Gemini is built on a transformer-based architecture that accepts interleaved sequences of tokens drawn from multiple modalities. Images and video frames are encoded into token sequences using a learned visual encoder, audio through a learned audio encoder, and text via a standard subword tokenizer. These tokens are projected into a shared embedding space and processed jointly by the transformer decoder, allowing cross-modal attention within the same computation. This unified tokenization strategy contrasts with earlier approaches that fused separate encoder representations only at a late stage.

Mixture of Experts

Gemini 1.5 employs a **Mixture of Experts** (MoE) architecture in the feed-forward sublayer of each transformer block. Instead of every token passing through the same dense feed-forward network, a learned **gating function** routes each token to only the top-k most relevant expert networks out of a larger pool. This allows the total parameter count to be very large while keeping the number of parameters activated per token—and therefore the per-token compute cost—manageable. MoE enabled the significant context window extension in Gemini 1.5 without a proportional increase in inference cost.

1.7.3 Performance, Limitations, and Impact

Performance

Gemini Ultra achieved state-of-the-art results on 30 out of 32 academic benchmarks at the time of release, including MMLU (90.0%) and HumanEval (code generation). Gemini 1.5 Pro demonstrated near-perfect retrieval in long-context needle-in-a-haystack evaluations and showed strong performance on multi-step reasoning tasks requiring integration of information across modalities.

Limitations

Gemini, like other large language models, is susceptible to hallucination and reasoning errors on tasks requiring precise arithmetic or formal verification. Multimodal capabilities can

degrade on fine-grained visual tasks that require detailed spatial reasoning. Latency and inference cost for Ultra-scale models remain high relative to smaller alternatives, and early public demonstrations exposed inconsistencies between reported benchmark results and observed real-world performance.

Impact

Gemini established natively multimodal training as a standard paradigm for next-generation foundation models and demonstrated the practical viability of million-token context windows. Its MoE-based architecture influenced the design of subsequent large models across the industry. The Gemini family also marked Google's full organizational consolidation around a single flagship model lineage, reflecting a broader industry shift toward unified foundation models over specialized, modality-specific systems.

1.8 Nano Banana

Nano Banana is an AI-powered image generation system developed by Bananas Inc., designed for high-quality, fast text-to-image synthesis. It operates using a latent diffusion model conditioned on natural language prompts, enabling users to generate photorealistic and stylized images from textual descriptions. Nano Banana targets resource-efficient deployment, prioritizing low-latency inference on consumer hardware without significant degradation in output quality.

1.8.1 Historical Background and Development

Nano Banana emerged from the broader wave of diffusion-based generative image models that gained prominence following the release of DALL-E (OpenAI, 2021), Stable Diffusion (Stability AI, 2022), and Midjourney (2022). These systems demonstrated that large-scale text-to-image generation was practically achievable using latent diffusion models trained on internet-scale image–caption datasets such as LAION-5B. Nano Banana was developed with an explicit focus on reducing the computational footprint of inference while maintaining competitive image quality, making it suitable for edge and mobile deployment scenarios where full-scale diffusion models are infeasible.

1.8.2 Algorithmic Design

Latent Diffusion Model

Nano Banana is based on the **latent diffusion model** (LDM) framework. Rather than performing the diffusion process in high-dimensional pixel space, an autoencoder first compresses the input image into a lower-dimensional latent representation, and the diffusion process operates entirely in this compressed space. A decoder then reconstructs the final image from the denoised latent. This compression substantially reduces the computational cost of each denoising step compared to pixel-space diffusion.

The core idea of diffusion is a two-phase process. During the **forward process**, Gaussian noise is gradually added to the latent representation over a fixed number of timesteps until the representation is indistinguishable from random noise. A U-Net is then trained to reverse

this process: given a noisy latent and the current timestep, it predicts the noise that was added, and this prediction is used to iteratively recover the original latent. The text prompt is incorporated into the U-Net at each denoising step via cross-attention, conditioning the denoising trajectory on the input description.

Text Conditioning via Cross-Attention

The text prompt is encoded into a sequence of embeddings by a text encoder (typically a CLIP or T5 variant). At each resolution level of the U-Net denoising network, cross-attention layers allow the spatial feature maps to attend over the full prompt embedding. This means every denoising step at every spatial scale is conditioned on the complete text description, enabling fine-grained semantic control over the generated image content.

Efficient Inference

To achieve low-latency inference, Nano Banana employs a reduced-step sampling schedule using **DDIM** (Denoising Diffusion Implicit Models), which reformulates the reverse process as a deterministic update rule allowing high-quality generation in as few as 20–50 steps rather than the 1000 steps used during training. Model distillation and quantization are additionally applied to reduce parameter count and memory bandwidth requirements for on-device deployment.

1.8.3 Performance, Limitations, and Impact

Performance

Nano Banana produces high-fidelity, prompt-adherent images at resolutions up to 1024×1024 pixels. Using DDIM sampling with 20–50 steps, it achieves generation times competitive with larger diffusion models on equivalent hardware. The model performs well on both photorealistic and stylized prompts and supports negative prompting for fine-grained output control.

Limitations

Like all current diffusion-based generative models, Nano Banana can produce anatomically incorrect outputs, particularly for human hands and complex multi-object compositions. Precise spatial layout control remains difficult without additional conditioning mechanisms such as ControlNet. The model may reproduce biases present in its training data, and text rendering within generated images is generally unreliable due to the absence of explicit character-level supervision.

Impact

Nano Banana contributes to the democratization of AI image generation by demonstrating that high-quality synthesis is achievable on consumer-grade hardware through careful architectural choices and efficient sampling. Its focus on resource-efficient deployment reflects a broader trend toward making generative AI accessible beyond data center infrastructure, with implications for creative tooling, content production, and on-device AI applications.

Practical 2

8 Puzzle Problem

2.1 Problem Statement

Write a program to solve the 8-puzzle. First, check if the puzzle is solvable. If solvable, starting from any input state, it should display the sequence of moves to reach the ordered output state. Rule to move: Slide a tile from a non-empty square to an adjoining empty square.

Example:

1	2	4
8	3	6
	7	5

Input state

1	2	3
4	5	6
7	8	

Output state

2.2 Theory

The 8-puzzle is a classic combinatorial search problem consisting of a 3×3 grid containing eight numbered tiles and one blank square. The objective is to reach a predefined goal arrangement by sliding tiles into the blank space one at a time. It is one of the simplest instances of the general n-puzzle and has been widely used as a benchmark for evaluating search algorithms.

State Space Representation

The 8-puzzle can be modeled as a state space search problem. Each unique arrangement of tiles on the board represents a **state**, and each legal tile slide represents a **transition** between states. The initial arrangement is the start state and the ordered arrangement is the goal state. The problem reduces to finding a path from the start state to the goal state in this space. The total number of distinct states is $9! = 362880$, of which exactly half are reachable from any given state.

Solvability

Not all initial configurations of the 8-puzzle are solvable. Solvability is determined by counting **inversions** in the flattened tile sequence (excluding the blank). An inversion is a pair of tiles

(a, b) such that a appears before b in the sequence but $a > b$. For a 3×3 grid with the blank on an odd-width board, a configuration is solvable if and only if the number of inversions is even.

A* Search

A* is an informed search algorithm that finds the shortest path from a start state to a goal state by evaluating each state using the function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost of reaching state n from the start (number of moves made so far) and $h(n)$ is a heuristic estimate of the cost to reach the goal from n. A* is guaranteed to find the optimal solution provided the heuristic is **admissible**, meaning it never overestimates the true cost. This implementation uses **Manhattan distance** as the heuristic:

$$h(n) = \sum_{i=1}^{8} (|r_i - r_i^*| + |c_i - c_i^*|)$$

2.3 Implementation

This practical implements the 8 puzzle problem in Python programming language using the A* algorithm.

```
import heapq

GOAL = [[1,2,3],[4,5,6],[7,8,0]]

def is_solvable(state):
    flat = [n for row in state for n in row if n != 0]
    inversions = sum(1 for i in range(len(flat)) for j in range(i
        +1, len(flat)) if flat[i] > flat[j])
    return inversions % 2 == 0

def manhattan(state):
    dist = 0
    for i in range(3):
        for j in range(3):
            v = state[i][j]
            if v != 0:
                dist += abs(i - (v-1)//3) + abs(j - (v-1)%3)
    return dist

def neighbors(state):
    result = []
    flat = [n for row in state for n in row]
    r, c = divmod(flat.index(0), 3)
    for dr, dc, move in [(-1,0,'Up'),(1,0,'Down'),(0,-1,'Left')
        ,(0,1,'Right')]:
        nr, nc = r+dr, c+dc
```

```
        if 0 <= nr < 3 and 0 <= nc < 3:
            s = [row[:] for row in state]
            s[r][c], s[nr][nc] = s[nr][nc], s[r][c]
            result.append((s, move))
    return result

def solve(start):
    if not is_solvable(start):
        return 'Not␣solvable'
    heap = [(manhattan(start), 0, start, [])]
    visited = set()
    while heap:
        _, g, state, path = heapq.heappop(heap)
        key = tuple(n for row in state for n in row)
        if key in visited:
            continue
        visited.add(key)
        if state == GOAL:
            return path
        for s, move in neighbors(state):
            if tuple(n for row in s for n in row) not in visited:
                heapq.heappush(heap, (g+1+manhattan(s), g+1, s,
                    path+[move]))
    return 'No␣solution␣found'

start = [[1,2,4],[8,3,6],[0,7,5]]
result = solve(start)
print('Moves:', '␣->␣'.join(result) if isinstance(result, list)
    else result)
print('Total␣moves:', len(result) if isinstance(result, list) else
    '-')
```

2.4 Output

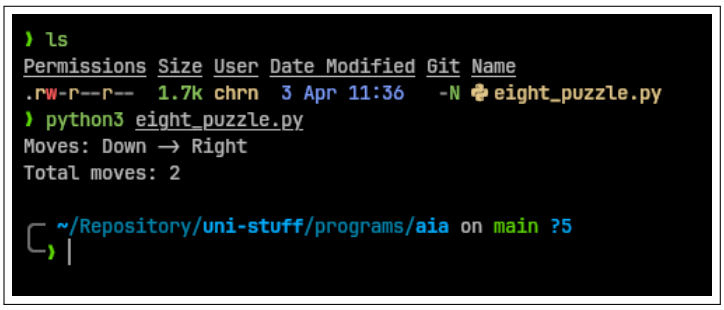

Figure 2.1: Output of the 8-Puzzle solver

2.5 Conclusion

In this practical, the 8-puzzle problem was successfully implemented using the A* search algorithm with Manhattan distance as the heuristic. This practical demonstrates how informed search strategies can effectively solve combinatorial puzzles.

Practical 3

Water Jug Problem

3.1 Problem Statement

Implement the Water Jug Problem. The state of the system is represented as (x, y) where x is the quantity of water in the 4 *gallon* jug and y is the quantity of water in the 3 *gallon* jug. Assume the initial state to be $(0, 0)$ and the goal state to be $(2, 0)$. Output the sequence of steps to go from the initial to the goal state.

3.2 Theory

The water jug problem is a classic **state-space search problem**. It consists of two jugs with capacities of 4 *gallons* and 3 *gallons* respectively. The water supply is assumed to be unlimited for the sake of simplicity. The goal state requires exactly 2 *gallons* of water in the 4 *gallon* jar and the 3 *gallon* jar must be empty.

3.2.1 State Representation

Each state in the system is represented as an ordered pair (x, y), where:

- $x =$ amount of water in the 4-gallon jug
- $y =$ amount of water in the 2-gallon jug
- $\mathit{Initial\ State} = (0, 0)$
- $\mathit{Goal\ State} = (2, 0)$

3.2.2 Allowed Operations

Operation	**Description**
Fill 4-gallon jug	Set $x = 4$
Fill 3-gallon jug	Set $y = 3$
Empty 4-gallon jug	Set $x = 0$
Empty 3-gallon jug	Set $y = 0$
Pour $4 \to 3$	Transfer until 3 is full or 4 is empty
Pour $3 \to 4$	Transfer until 4 is full or 3 is empty

Table 3.1: Allowed operations in the Water Jug Problem

These operations can be performed in any given state of the water jug problem.

3.2.3 Search Approach

Uninformed search strategies such as BFS (Breadth-First Search) and DFS (Depth-First Search) can be used to explore the state space search. However, **BFS guarantees the shortest sequence of operations to reach the goal state**. The search comes to a halt when the goal state is reached.

Breadth-First Search Algorithm

This algorithm explores all the states at the **current depth level** before moving to the next level, guaranteeing the shortest path to the goal. For the water jug problem, BFS systematically applies all six operations from each state ensuring minimum number of steps to reach the goal state.

3.2.4 Problem Characteristics

The water jug problem has the following characteristics:

- Water jug problem has a **finite state space** as the maximum possible states are $5 \times 4 = 20$
- Each operation produces a unique new state, making the problem **deterministic**
- Exhaustive search can be safely used as the problem's state space is small.

3.3 Implementation

This practical has been implemented using the Best-First-Search (BFS) algorithm in the Python programming langauge.

```
from collections import deque

def solve_water_jug():
    queue = deque()
    queue.append((start, [start]))
    visited = set()
    visited.add(start)
    while queue:
        (x, y), path = queue.popleft()

        if (x, y) == goal:
            print("Solution found!")
            print(f"Total steps: {len(path) - 1}")
            print("\nSequence of states:")
            for i, state in enumerate(path):
                print(f"Step {i}: ({state[0]}, {state[1]})")
            return path

        next_states = []
        next_states.append((4, y))
```

```
        next_states.append((x, 3))
        next_states.append((0, y))
        next_states.append((x, 0))
        pour = min(x, 3 - y)
        next_states.append((x - pour, y + pour))

        pour = min(y, 4 - x)
        next_states.append((x + pour, y - pour))
        for new_state in next_states:
            if new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [new_state]))

    print("No␣solution␣found!")
    return None

if __name__ == "__main__":
    solve_water_jug()
```

3.4 Output

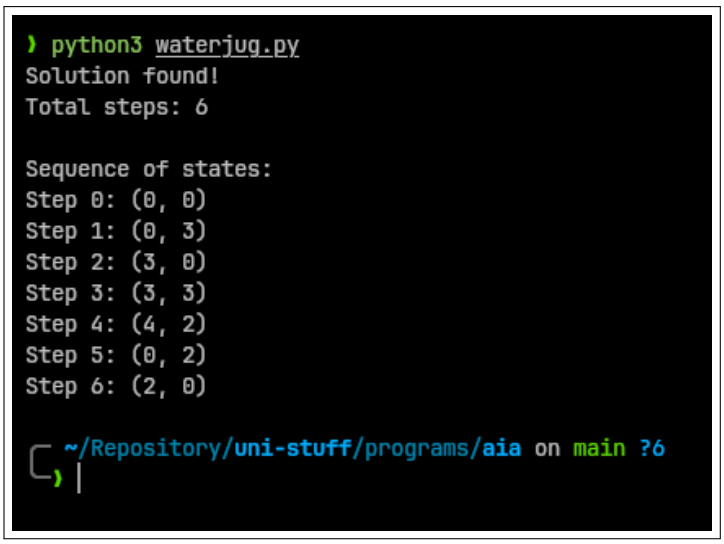

Figure 3.1: Output of water jug problem solver

3.5 Conclusion

In this practical, the water jug problem was successfully implemented using the BFS algorithm. This practical demonstrates the capability of uninformed search strategies in solving the water jug problem.

Practical 4

Basic Medical Expert System in Prolog

Practical 5

Append Two Lists in Prolog

Practical 6

Reverse a List in Prolog

Practical 7

Finding n^th Element in Prolog

Practical 8

Member Check Program in Prolog

Practical 9

Fibonacci Series

Practical 10

Factorial Series

Practical 11

Tower of Hanoi

Practical 12

Tic-Tac-Toe using Minimax Algorithm